EP6. The ellipsoid method for linear programming

Lidivine Kengne | IE 5311-001 (D01), Homework 5

1. The main idea. The method never reads a constraint matrix. It is handed a convex set $K \subseteq \mathbb{R}^n$ through a *separation oracle*: a routine that, for any query z, either certifies $z \in K$ or returns $d \neq 0$ with $d^\top x < d^\top z$ for all $x \in K$. That is the whole interface, and the whole point: K may have exponentially many facets, none ever written down. Write $E(z, Q) = \{x : (x - z)^\top Q^{-1}(x - z) \leq 1\}$ with $Q \succ 0$ and start from any $E_0 \supseteq K$. Query the oracle at the centre z_k: if $z_k \in K$ the search ends, otherwise the returned d confines K to the half ellipsoid $E_k \cap \{x : d^\top x \leq d^\top z_k\}$, and E_{k+1} is the smallest ellipsoid containing that. With $g = Q_k d / \sqrt{d^\top Q_k d}$,

$$z_{k+1} = z_k - \frac{g}{n+1}, \qquad Q_{k+1} = \frac{n^2}{n^2-1}\Big(Q_k - \frac{2}{n+1}\, g\, g^\top\Big),$$

a rank-one update costing $O(n^2)$ arithmetic. The volume contracts by a fixed factor whatever the cut, so a point of K is found within $O(n \log(V/v))$ calls when $\mathrm{vol}(K) \geq v$ and $\mathrm{vol}(E_0) \leq V$. That factor is usually quoted as $e^{-1/(2(n+1))}$, but that is only a bound. Since $g^\top Q_k^{-1} g = 1$ by construction, $\det\big(Q_k - \frac{2}{n+1} g g^\top\big) = \det Q_k \cdot \frac{n-1}{n+1}$, so

$$\frac{\mathrm{vol}(E_{k+1})}{\mathrm{vol}(E_k)} = \Big[\Big(\tfrac{n^2}{n^2-1}\Big)^n \tfrac{n-1}{n+1}\Big]^{1/2},$$

a constant attained *exactly* at every step, whatever the cut. For $n = 3$ it is $27/32 = 0.84375$ against the bound $e^{-1/8} = 0.8825$. The contraction still weakens as n grows — each cut discards about a $1/n$ fraction — which is why the method is slow.

2. Historical significance: linear programming is in P. For an LP with rational data of bit size L, a nonempty feasible region can be perturbed to have volume at least $2^{-O(nL)}$ and lies in a ball of radius $2^{O(L)}$, so the trivial oracle "scan the rows, return a violated one" gives $O(n^2 L)$ iterations. Khachiyan's 1979 proof was the first of its kind and a surprise: simplex was then as now the only algorithm anyone ran, and it is exponential in the worst case (Klee and Minty's distorted cube forces Dantzig's rule through all 2^n vertices).

3. The lasting significance: separation equals optimization. Because the method touches K only through the oracle, its running time depends on the number of *variables* and the cost of one separation, never on the number of constraints. Grötschel, Lovász and Schrijver turned this into an equivalence: for a well-described polyhedron, optimizing a linear function over it and separating from it are polynomial-time equivalent. An exponential family of constraints is therefore no obstacle, given a polynomial oracle.

4. Where this course has already used the idea. The DFJ subtour constraints number $2^{|V|}$, and the lazy callback written for Homework 1 is a separation oracle for them, finding the violated subtour by a minimum cut. Homework 5 is the same idea from the other side: the Dantzig–Wolfe master holds one column per pattern, so its *dual* holds one row per pattern, and the pricing knapsack is precisely the separation oracle for that dual. Column generation is a cutting-plane method on the dual, and the ellipsoid method would solve the same relaxation in polynomial time from the same oracle — which is how one proves such a relaxation tractable despite exponentially many variables.

5. What it costs, measured. Run on the Part 1 LP, with the oracle scanning the rows of $Ax \leq b$, $x \geq 0$ and $c^\top x \geq z^\star - \delta$, starting from a ball of radius 20 (`ep6_ellipsoid.py`):

δ	oracle calls	$c^\top x$ found	observed ratio/step
10^0	29	23.807686	0.843750
10^{-1}	69	23.942739	0.843750
10^{-2}	105	23.992071	0.843750
10^{-3}	158	23.999703	0.843750
10^{-4}	182	23.999901	0.843750

The ratio is 0.84375 to six figures every run, as the derivation requires. Since $\mathrm{vol}(K) \sim \delta^n$, a decade of δ costs a factor 10^3 of volume: 40.7 calls at the exact rate, 55.3 at the bound, against $40, 36, 53, 24$ observed. The bound overstates the work by about a third and the method is still hopeless — four digits cost 182 oracle calls in $\mathbb{R}^3$, where the simplex of Part 2 reached $x^\star = (4, 4, 4)$ exactly in 3 pivots. Its standing is as an existence proof and a reduction tool.